

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

CS 401/MCS 401 Lecture 21
Computer Algorithms I
Jan Verschelde, 5 August 2026

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

final exam policies

- The Final Exam happens on Friday 7 August, from 10AM till 11:40AM.
- The exam covers all lectures.
This corresponds to the first eight chapters in the textbook.
Sections not explicitly covered in class will not be on the exam.
- The sample questions represent the type of questions.
You may **not** assume that the range of topics is limited to the algorithms covered by the questions on this page.

Review for the Final Exam

1 Policies and Overview

- final exam policies
- **overview of topics**

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

worst case running times and big $O()$

The running time of an algorithm is expressed as a function of the input size, often expressed by the dimensions of the problem.

Because we are mainly interested in asymptotic worst case behaviour, the big $O()$ notation is used for the upper bound on the running time. $\Omega()$ is for lower bounds, and $\Theta()$ for both lower and upper.

An implementation of an algorithm is done by selecting data structures so that all operations in the algorithm are fully specified.

An algorithm is efficient if its running time is bounded by a polynomial function in the input size.

greedy algorithms

A greedy algorithm solves an optimization method locally, applying a greedy rule in each step.

Optimality is proven via an exchange argument.

We have greedy algorithms for interval scheduling (one resource) and for the interval partitioning problem (multiple resources) (L-5).

Other applications of greedy algorithms include:

- 1 scheduling to minimize maximum lateness (L-6),
- 2 optimal caching (L-6),
- 3 Dijkstra's algorithm for shortest paths (L-6), and
- 4 Kruskal, Prim, reverse-delete for minimum spanning trees (L-7).

divide and conquer

Divide and conquer breaks up the problem in smaller parts, solves each subproblem recursively, and combines the solutions of the subproblems.

Counting inversions in a sequence of numbers, using merge sort leads to subquadratic or $O(n \log(n))$ times for n numbers.

Applications of divide and conquer include

- 1 finding the closest pair of points (L-8),
- 2 Karatsuba's integer multiplication (L-8),
- 3 the fast Fourier transform (L-9), and
- 4 Strassen's method for matrix multiplication (L-9).

To derive the subcubic running time of Strassen's method, the master theorem was applied.

dynamic programming

Dynamic programming must be applied when greedy rules fail.

A recurrence is developed to explore the problem space, which is then elaborated with memoization.

We applied dynamic programming to the following problems:

- 1 weighted interval scheduling (L-12),
- 2 segmented least squares (L-13),
- 3 sequence alignment (L-13),
- 4 subset sum and knapsack with natural numbers as weights (L-14),
- 5 shortest paths in graphs with negative weights (L-14).

Dynamic programming organizes a systematic search to solve all NP-hard problems covered in the last four lectures.

network flow

The Ford-Fulkerson algorithm computes the maximum flow in weighted directed graph, where weights on the edges are capacities.

The maximum flow corresponds to the minimum cut in the network.

The running time is $O(mC)$ for m the number of edges and C the bound on the maximum flow, computed from the sum of the capacities of the edges leaving the source; or pseudo-polynomial time.

We discussed five applications:

- 1 the bipartite matching problem (L-16),
- 2 circulation with demands (L-16),
- 3 disjoint paths in graphs (L-17),
- 4 the survey design problem (L-17), and
- 5 airline scheduling (L-17).

polynomial-time reductions

The circuit satisfiability problem is NP-complete,
as shown by the generality of circuits to encode any algorithm.
3-Satisfiability is polynomial-time reducible to circuit satisfiability.

We reduced 3-SAT four times:

- 1 $3\text{-SAT} \leq_P \text{independent set (L-18)}$
vertex cover $\leq_P$ independent set
vertex cover $\leq_P$ set cover
- 2 $3\text{-SAT} \leq_P \text{circuit satisfiability (L-19)}$
- 3 $3\text{-SAT} \leq_P \text{Hamiltonian cycle (L-20)}$
Hamiltonian cycle $\leq_P$ traveling salesman
- 4 $3\text{-SAT} \leq_P \text{3-dimensional matching (L-19)}$
3-dimensional matching $\leq_P$ to subset sum (L-20)

Each reduction is defined by a polynomial-time algorithm.

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- **cost functions**
- minimum spanning trees
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

cost functions

Prove that $1 + 2n + 3n^2 + 4n^3$ is $O(n^3)$.

asymptotic growth

Prove that $1 + 2n + 3n^2 + 4n^3$ is $O(n^3)$.

Apply the following rules:

- 1 Any monomial of lower degree is bounded by any monomial of higher degree multiplied by a positive constant.
- 2 If g is $O(f)$ and h is $O(f)$, then $g + h$ is $O(f)$.
- 3 $4n^3$ is $O(n^3)$ as 4 is just another constant.

Alternatively, compute the following limit

$$\lim_{n \rightarrow \infty} \frac{1 + 2n + 3n^2 + 4n^3}{n^3} = 4$$

which yields the stronger bound $\Theta(n^3)$.

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- **minimum spanning trees**
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

minimum spanning trees

Let G be an undirected weighted graph.

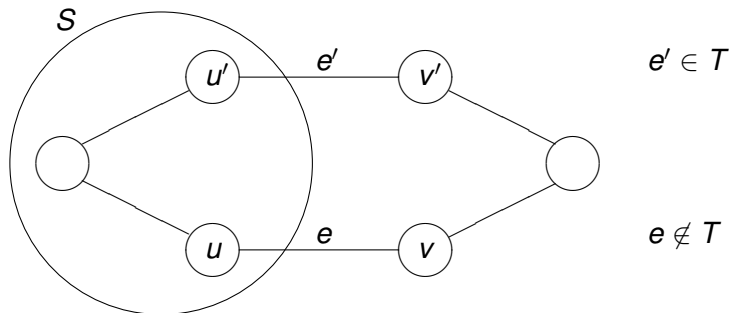
Let (u, v) be the edge with minimum weight.

Is (u, v) contained in some minimum spanning tree of G ?

Justify your answer.

an exchange argument

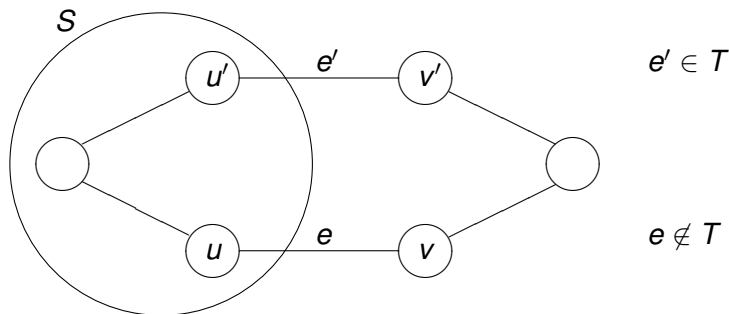
Let (u, v) the edge with minimum weight, and consider a minimum spanning tree T which would not contain (u, v) .



Because T is a spanning tree, u and v are connected via a path that does not use the edge $e = (u, v)$, but uses e' .

the exchange argument continued

Let (u, v) the edge with minimum weight, and consider a minimum spanning tree T which would not contain (u, v) .



If the weight on e is the same on e' , then indeed, T can be a minimum spanning tree. Otherwise, we could exchange e' with e and obtain a spanning tree with smaller cost. Then we have a contradiction.

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- **divide and conquer**
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

divide and conquer

Two indices $i < j$ in a sequence $a_1, a_2, \dots, a_n$ of numbers form an inversion if $a_i > a_j$.

Consider the sequence 19, 73, 78, 63, 26, 95, 58, 27.

- 1 Illustrate how to compute the number of inversions in the sequence in time $O(n \log(n))$.
- 2 While the number of inversions can be as large as quadratic in n , the running time to count the inversions is subquadratic. Explain why this is so.

inversions in 19, 73, 78, 63, 26, 95, 58, 27

Abbreviate Sort-and-Count by S.

$$\begin{aligned} &S(19, 73, 78, 63, 26, 95, 58, 27) \\ &= S(19, 73, 78, 63) + S(26, 95, 58, 27) \\ &= S(19, 73) + S(78, 63) + S(26, 95) + S(58, 27) \\ &= (0, (19, 73)) + (1, (63, 78)) + (0, (26, 95)) + (1, (27, 58)) \\ &= (0 + 1 + 1, (19, 63, 73, 78)) + (0 + 1 + 1, (26, 27, 58, 95)) \\ &= (3, (19, 63, 73, 78)) + (2, (26, 27, 58, 95)) \\ &= (3 + 2 + 3 + 3 + 3, (19, 26, 27, 58, 63, 73, 78, 95)) \\ &= (14, (19, 26, 27, 58, 63, 73, 78, 95)) \end{aligned}$$

a subquadratic algorithm

The algorithm follows merge sort, which has a running time of $O(n \log(n))$, on a sequence of n elements.
 $O(n \log(n))$ is subquadratic.

The inversions are computed in the merge stage of the sort.

There can be at most one addition for every element.

The merge stage of merge sort has running time $O(n)$ and making at most one addition per element does not increase the $O(n)$ running time.

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- NP-completeness

CS 401/MCS 401 Lecture 21
Computer Algorithms I
Jan Verschelde, 5 August 2026

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- **dynamic programming**
- the Ford-Fulkerson algorithm
- NP-completeness

dynamic programming

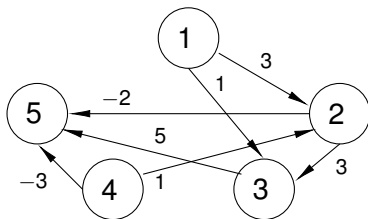
Consider a weighted directed graph with 5 vertices and weights on the edges

$$w_{(2,5)} = -2, w_{(4,2)} = 1, w_{(2,4)} = 2, w_{(1,2)} = 3, w_{(1,3)} = 1, \\ w_{(3,5)} = 5, w_{(2,3)} = 3, w_{(4,5)} = -3.$$

Our problem is to compute the minimum cost path from 1 to 5.

- 1 Explain why Dijkstra's algorithm does not work for this type of problem.
- 2 What kind of algorithm will you use?
- 3 Illustrate the execution of the algorithm on this graph.

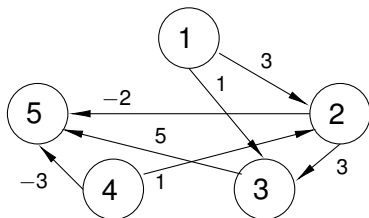
why Dijkstra's algorithm does not work



The shortest path from 1 to 5 is 1, 2, 5, with length $3 - 2 = 1$.

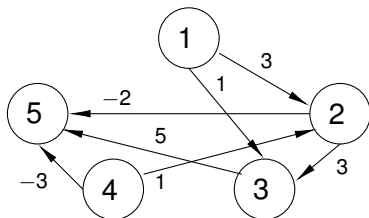
Dijkstra's algorithm selects 3 as the next vertex from 1.
And then the path to 5 via 3 has length 6.

the Bellman-Ford algorithm – step 0



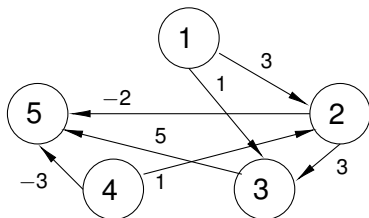
#edges	vertices				
	1	2	3	4	5
5					
4					
3					
2					
1					
0	∞	∞	∞	∞	0

the Bellman-Ford algorithm – step 1



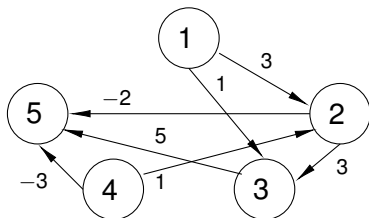
	vertices				
#edges	1	2	3	4	5
5					
4					
3					
2					
1	∞	-2	5	-3	0
0	∞	∞	∞	∞	0

the Bellman-Ford algorithm – step 2



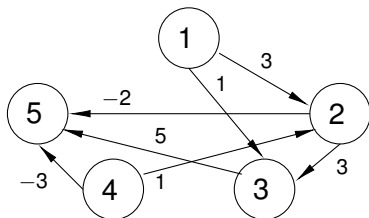
	vertices				
#edges	1	2	3	4	5
5					
4					
3					
2	1	-2	5	-3	0
1	∞	-2	5	-3	0
0	∞	∞	∞	∞	0

the Bellman-Ford algorithm – step 3



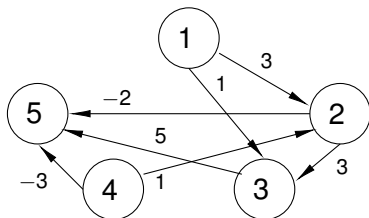
	vertices				
#edges	1	2	3	4	5
5					
4					
3	1	-2	5	-3	0
2	1	-2	5	-3	0
1	∞	-2	5	-3	0
0	∞	∞	∞	∞	0

the Bellman-Ford algorithm – step 4



#edges	vertices				
	1	2	3	4	5
5					
4	1	-2	5	-3	0
3	1	-2	5	-3	0
2	1	-2	5	-3	0
1	∞	-2	5	-3	0
0	∞	∞	∞	∞	0

the Bellman-Ford algorithm – step 5



#edges	vertices				
	1	2	3	4	5
5	1	-2	5	-3	0
4	1	-2	5	-3	0
3	1	-2	5	-3	0
2	1	-2	5	-3	0
1	∞	-2	5	-3	0
0	∞	∞	∞	∞	0

Review for the Final Exam

1 Policies and Overview

- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- dynamic programming
- **the Ford-Fulkerson algorithm**
- NP-completeness

the Ford-Fulkerson algorithm

Consider the flow network with source s , sink t , and internal nodes a , b , c , and d . The capacities on the edges are

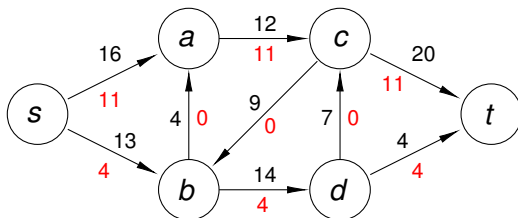
$$c_{(s,a)} = 16, c_{(s,b)} = 13, c_{(a,c)} = 12, c_{(b,a)} = 4, c_{(c,b)} = 9, \\ c_{(b,d)} = 14, c_{(d,c)} = 7, c_{(c,t)} = 20, c_{(d,t)} = 4,$$

and flow defined by

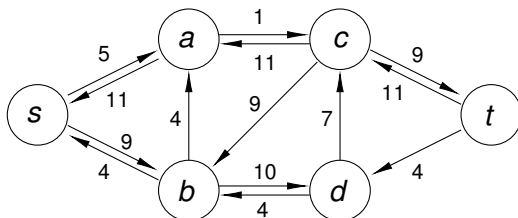
$$f_{(s,a)} = 11, f_{(s,b)} = 4, f_{(a,c)} = 11, f_{(b,a)} = 0, f_{(c,b)} = 0, \\ f_{(b,d)} = 4, f_{(d,c)} = 0, f_{(c,t)} = 11, f_{(d,t)} = 4.$$

- 1 Draw the flow network and show one step in the Ford-Fulkerson algorithm.
- 2 What is the greatest increase in flow you can find? Give the value of the increase.

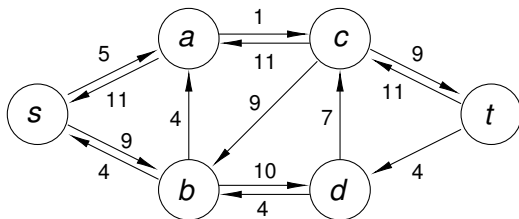
computing the residual graph



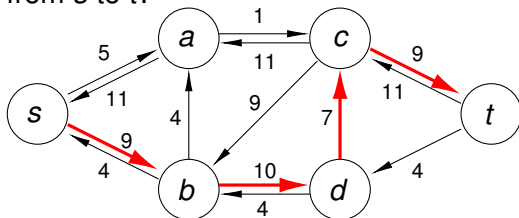
The residual graph:



finding the greatest increase in flow



Finding a path from s to t :



The bottleneck is 7, which is the greatest increase in the flow.

bipartite matching

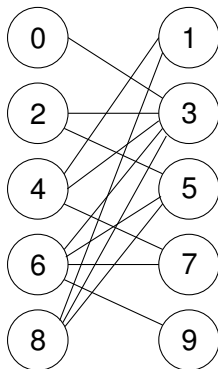
Consider the bipartite graph $G = (V, E)$, defined by $V = X \cup Y$, $X = \{0, 2, 4, 6, 8\}$, $Y = \{1, 3, 5, 7, 9\}$, and $E = \{(0, 3), (2, 3), (2, 5), (4, 1), (4, 3), (4, 7), (6, 3), (6, 5), (6, 7), (6, 9), (8, 1), (8, 3), (8, 5)\}$.

Use this example to explain the application of the Ford-Fulkerson algorithm to compute a bipartite matching in this graph.

- 1 Setup the flow network. Show how the maximum flow in the network corresponds to a bipartite matching.
- 2 What is the running time of this algorithm?

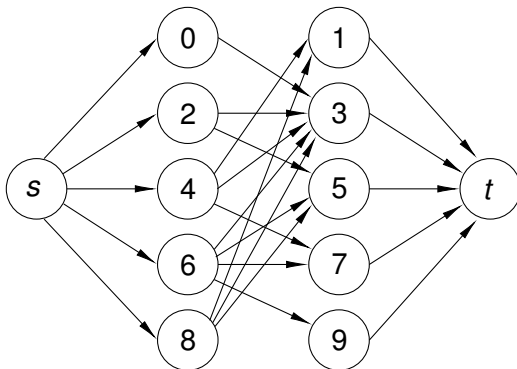
a bipartite graph

$X = \{0, 2, 4, 6, 8\}$, $Y = \{1, 3, 5, 7, 9\}$, and $E = \{(0, 3), (2, 3), (2, 5), (4, 1), (4, 3), (4, 7), (6, 3), (6, 5), (6, 7), (6, 9), (8, 1), (8, 3), (8, 5)\}$.



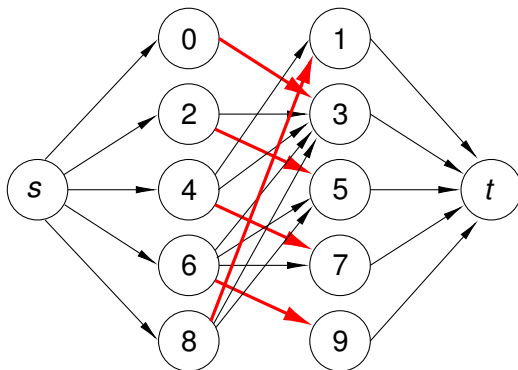
flow network for a bipartite matching

$X = \{0, 2, 4, 6, 8\}$, $Y = \{1, 3, 5, 7, 9\}$, and $E = \{(0, 3), (2, 3), (2, 5), (4, 1), (4, 3), (4, 7), (6, 3), (6, 5), (6, 7), (6, 9), (8, 1), (8, 3), (8, 5)\}$.



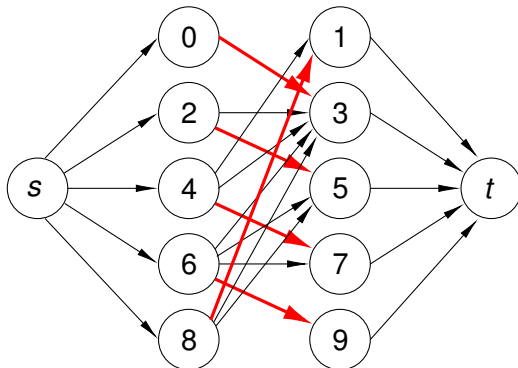
All capacities of the edges are one.

maximum flow network and matchings



Because all capacities of the edges are one, after selecting an edge (i, j) , $i \in X$, $j \in Y$, neither i nor j can occur in any other selection, because this would violate the conservation conditions on the flow. All edges with flow define the matching.

running time



The running time is $O(mn)$ for n vertices and m edges, because the bound on the maximum flow is n ($n/2$ is sharp).

Review for the Final Exam

1 Policies and Overview

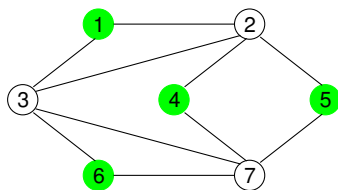
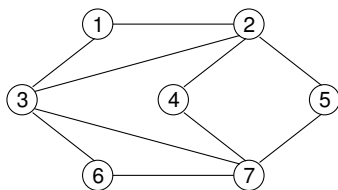
- final exam policies
- overview of topics

2 Sample Questions

- cost functions
- minimum spanning trees
- divide and conquer
- dynamic programming
- the Ford-Fulkerson algorithm
- **NP-completeness**

the independent set problem

Consider the graph below with independent vertices in green.

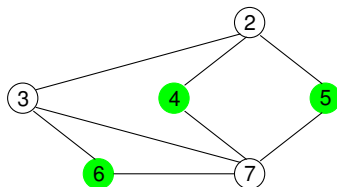
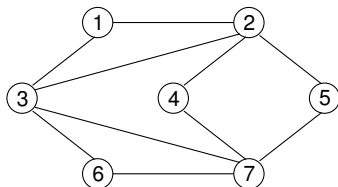


Apply dynamic programming to the independent set problem.

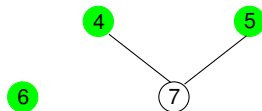
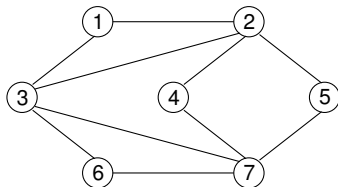
- 1 Develop a recurrence to derive the algorithm.
- 2 Given k , the size of an independent set, describe the algorithm to compute an independent set of size k , or to return nothing if there is no independent set of size k .
- 3 What is the running time of this algorithm?

removing a vertex

Removing a vertex which belongs to an independent set:



Removing a vertex and all its adjacent vertices:



developing a recurrence

Let k be the size of the largest independent set in $G = (V, E)$.

For $v \in V$, define $G_1 = (V_1, E_1)$, $V_1 = V \setminus \{v\}$,

and $E_1 = E \setminus \{ (u, w) \mid u = v \text{ or } w = v \}$.

Let k_1 be the size of the largest independent set in G_1 .

If v is in the independent set, then we have to remove its neighborhood. Define the neighborhood of v as

$N_v = \{ u \in V \mid (u, v) \in E \text{ or } (v, u) \in E \}$. Define $G_2 = (V_2, E_2)$,

$V_2 = V_1 \setminus N_v$, and $E_2 = E_1 \setminus \{ (u, w) \mid u \in N_v \text{ or } w \in N_v \}$.

Let k_2 be the size of the largest independent set in G_2 .

Then, $k = \max(k_1, k_2 + 1)$.

a recursive algorithm

`size_largest_independent_set( $G = (V, E)$ )`

if $E = \emptyset$ then

return 0

else

let $v \in V$ with $N_v \neq \emptyset$

define $G_1 = (V_1, E_1)$, $V_1 = V \setminus \{v\}$

define $G_2 = (V_2, E_2)$, $V_2 = V_1 \setminus N_v$

$k_1 := \text{size_largest_independent_set}(G_1)$

$k_2 := \text{size_largest_independent_set}(G_2)$

return $\max(k_1, k_2 + 1)$

cost of the algorithm

Let $T(n)$ be the time on a graph with n vertices.

- ① G_1 has $n - 1$ vertices,
- ② G_2 can have at most $n - 2$ vertices, if $\#N_v = 1$,

thus:

$$T(n) = T(n - 1) + T(n - 2) + O(n),$$

where the $O(n)$ is the cost of computing N_v .

We recognize in $T(n)$ the Fibonacci recurrence.

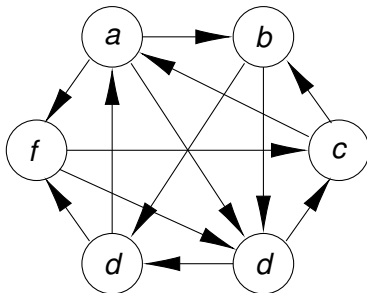
In the solution of the Fibonacci recurrence:

$$\left(\frac{1 + \sqrt{5}}{2} \right)^n$$

which is $O(1.62^n)$, an exponential running time.

Hamiltonian Cycle as Hamiltonian Path problem

Encode



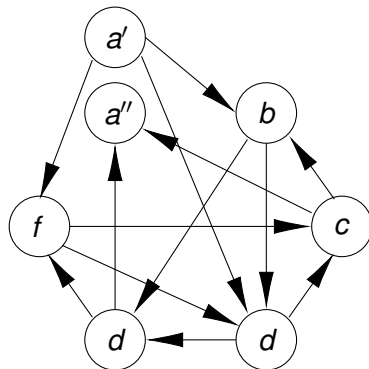
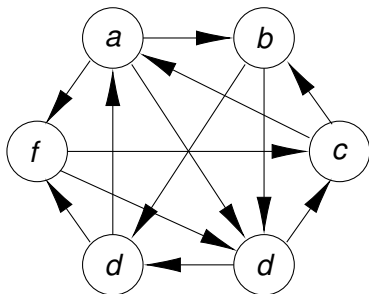
as a Hamiltonian path problem. Demonstrate that the solution of the Hamiltonian path problem solves the Hamiltonian cycle problem.

Why is this a polynomial-time reduction?

encoding Hamiltonian Cycle

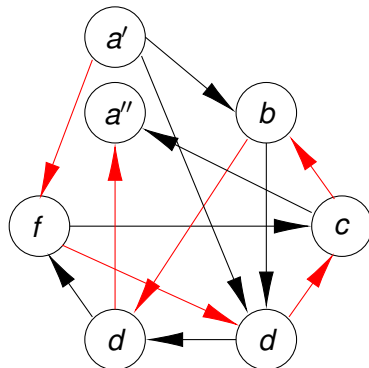
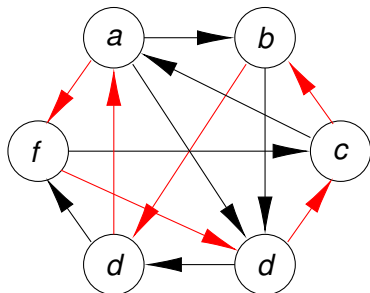
We have a cycle starting at a and ending at a .

Replacing a by a' and a'' in the graph at the left gives the graph at the right:



equivalent solutions

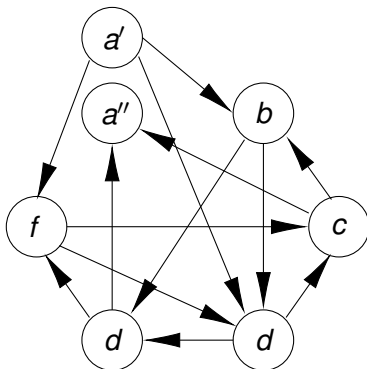
We have a cycle starting at a and ending at a , shown at the left:



and, at the right, we have a Hamiltonian path from a' to a'' .

a polynomial-time reduction

The graph G' derived from G , input to Hamiltonian cycle



has one extra vertex and the same number of edges as G .
 G' is constructed in $O(1)$ time, which is polynomial time.